

Java vs Python — Detailed Cheat Sheet for Java Developers

Author: Naresh Kumar Sanda

Audience: Senior Java Developers transitioning to Python

1. Mindset Shift — Java vs Python

Topic	Java	Python
Typing	Statically Typed	Dynamically Typed
Compilation	Compiled to JVM Bytecode	Interpreted
Verbosity	More Boilerplate	Minimal Syntax
Performance	Faster	Slightly Slower
Use Cases	Enterprise Apps	AI/ML, Automation, Scripting, Data Engineering
OOP	Strict OOP	Multi-paradigm
Memory Mgmt	JVM GC	Python GC
Blocks	Curly Braces {}	Indentation
File Naming	Class Name Based	Flexible
Null Handling	null	None
Access Modifiers	public/private/protected	Convention Based
Package Mgmt	Maven/Gradle	pip/poetry
Dependency Mgmt	pom.xml/build.gradle	requirements.txt/pyproject.toml

2. Hello World

Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

```
}  
}
```

Python

```
print("Hello World")
```

3. Variables and Data Types

Concept	Java	Python
Integer	int a = 10;	a = 10
Long	long a = 10L;	a = 10
Double	double d = 10.5;	d = 10.5
Boolean	boolean b = true;	b = True
String	String s = "abc";	s = "abc"
Character	char c = 'A';	c = 'A'
Null	null	None

Python Type Checking

```
x = 10  
print(type(x))
```

Java Type Checking

```
System.out.println(obj instanceof String);
```

4. Type Conversion

Java

```
int a = Integer.parseInt("10");  
String s = String.valueOf(a);
```

Python

```
x = int("10")  
s = str(x)
```

5. Input Handling

Java

```
Scanner sc = new Scanner(System.in);  
String name = sc.nextLine();
```

Python

```
name = input("Enter Name: ")
```

6. Operators

Operation	Java	Python
Equality	==	==
Reference Equality	==	is
AND	&&	and
OR		or
NOT	!	not

Operation	Java	Python
Modulus	%	%
Power	Math.pow(a,b)	a ** b

7. Conditional Statements

Java

```
if(age > 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Python

```
if age > 18:  
    print("Adult")  
else:  
    print("Minor")
```

8. Loops

For Loop

Java

```
for(int i=0; i<5; i++) {  
    System.out.println(i);  
}
```

Python

```
for i in range(5):  
    print(i)
```

While Loop

Java

```
while(i < 5) {  
    i++;  
}
```

Python

```
while i < 5:  
    i += 1
```

9. Functions / Methods

Java

```
public static int add(int a, int b) {  
    return a + b;  
}
```

Python

```
def add(a, b):  
    return a + b
```

Default Arguments

Python

```
def greet(name="Guest"):  
    print(name)
```

10. Collections Overview

Java	Python
Array	list
ArrayList	list
HashSet	set
HashMap	dict
Queue	deque
Stack	list
Stream API	list comprehensions/generator

11. Arrays vs Lists

Java Array

```
int[] arr = {1,2,3};
```

Python List

```
arr = [1,2,3]
```

Add Elements

Java

```
list.add(10);
```

Python

```
arr.append(10)
```

12. List Operations

Python

```
nums = [1,2,3]

nums.append(4)
nums.remove(2)
nums.sort()
nums.reverse()
```

Java Equivalent

```
List<Integer> nums = new ArrayList<>();
nums.add(4);
Collections.sort(nums);
Collections.reverse(nums);
```

13. Tuples (No Direct Java Equivalent)

```
point = (10, 20)
print(point[0])
```

Characteristics:

- Immutable
- Faster than list
- Used in data processing

14. Sets

Java

```
Set<Integer> set = new HashSet<>();
```

Python

```
s = {1,2,3}
s.add(4)
```

15. Maps / Dictionaries

Java

```
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
```

Python

```
data = {
    "a": 1,
    "b": 2
}
```

Iteration

Java

```
for(Map.Entry<String,Integer> e : map.entrySet()) {
    System.out.println(e.getKey());
}
```

Python

```
for k, v in data.items():
    print(k, v)
```

16. String Handling

Java

```
String name = "Nova";  
name.toUpperCase();
```

Python

```
name = "Nova"  
name.upper()
```

Common Operations

Operation	Java	Python
Length	str.length()	len(str)
Substring	substring()	slicing
Upper	toUpperCase()	upper()
Lower	toLowerCase()	lower()
Replace	replace()	replace()
Split	split()	split()
Contains	contains()	in

17. String Formatting

Java

```
String msg = String.format("Hello %s", name);
```

Python

```
msg = f"Hello {name}"
```

18. Exception Handling

Java

```
try {  
    int a = 10/0;  
} catch(Exception e) {  
    e.printStackTrace();  
} finally {  
    System.out.println("Done");  
}
```

Python

```
try:  
    a = 10/0  
except Exception as e:  
    print(e)  
finally:  
    print("Done")
```

19. Classes and Objects

Java

```
public class Employee {  
    private String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
}
```

Python

```
class Employee:
```

```
def __init__(self, name):  
    self.name = name
```

20. Access Modifiers

Java	Python Equivalent
private	<code>_variable</code> (convention)
protected	<code>__variable</code> (name mangling)
public	default

21. Inheritance

Java

```
class Animal {  
    void sound() {}  
}  
  
class Dog extends Animal {  
}
```

Python

```
class Animal:  
    def sound(self):  
        pass  
  
class Dog(Animal):  
    pass
```

22. Interfaces vs Abstract Classes

Java Interface

```
interface Payment {  
    void pay();  
}
```

Python

```
from abc import ABC, abstractmethod  
  
class Payment(ABC):  
  
    @abstractmethod  
    def pay(self):  
        pass
```

23. Constructors

Java

```
public Employee() {}
```

Python

```
def __init__(self):  
    pass
```

24. Static Methods

Java

```
public static void test() {}
```

Python

```
@staticmethod  
def test():  
    pass
```

25. Lambda Expressions

Java

```
list.stream()  
    .filter(x -> x > 5)  
    .forEach(System.out::println);
```

Python

```
nums = [1,2,3,6,7]  
result = list(filter(lambda x: x > 5, nums))
```

26. Streams vs Pythonic Style

Java

```
List<Integer> result = nums.stream()  
    .map(x -> x * 2)  
    .collect(Collectors.toList());
```

Python

```
result = [x * 2 for x in nums]
```

27. List Comprehensions

```
squares = [x*x for x in range(10)]
```

Equivalent Java logic usually needs loops or streams.

28. Generators

```
def generate_numbers():  
    for i in range(5):  
        yield i
```

Java Equivalent:

- Streams
 - Iterators
 - Reactive Streams
-

29. File Handling

Java

```
Files.write(Paths.get("a.txt"), "Hello".getBytes());
```

Python

```
with open("a.txt", "w") as f:  
    f.write("Hello")
```

30. JSON Handling

Java (Jackson)

```
ObjectMapper mapper = new ObjectMapper();
```

Python

```
import json

json.dumps(data)
json.loads(text)
```

31. Date and Time

Java

```
LocalDate.now();
```

Python

```
from datetime import datetime

print(datetime.now())
```

32. Multithreading

Java

```
Thread t = new Thread(() -> {
    System.out.println("Running");
});
```

Python

```
import threading

thread = threading.Thread(target=my_func)
thread.start()
```

33. Async Programming

Java

```
CompletableFuture.supplyAsync(() -> "Hello");
```

Python

```
import asyncio

async def main():
    print("Hello")
```

34. Database Access

Java JDBC

```
Connection con = DriverManager.getConnection(url);
```

Python

```
import sqlite3

con = sqlite3.connect("test.db")
```

35. REST APIs

Java Spring Boot

```
@RestController
public class TestController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello";
    }
}
```

Python Flask

```
from flask import Flask

app = Flask(__name__)

@app.route('/hello')
def hello():
    return "Hello"
```

36. Dependency Management

Java

- Maven
- Gradle

Python

- pip
- poetry
- pipenv

Install Package

```
pip install pandas
```

Equivalent Java:

```
<dependency>  
</dependency>
```

37. Virtual Environments

Python

```
python -m venv venv
```

Activate:

```
source venv/bin/activate
```

Windows:

```
venv\Scripts\activate
```

38. Important Python Packages for Java Developers

Area	Java	Python
Web	Spring Boot	Flask/FastAPI/Django
JSON	Jackson	json/pydantic
ORM	Hibernate	SQLAlchemy
Data Processing	Spark	pandas/polars
AI/ML	DL4J	scikit-learn/tensorflow/pytorch
REST Calls	RestTemplate/WebClient	requests/httpx
Async	CompletableFuture	asyncio
Unit Test	JUnit	pytest

Area	Java	Python
Logging	log4j/slf4j	logging
Scheduling	Quartz	APScheduler

39. Logging

Java

```
private static final Logger log = LoggerFactory.getLogger(Test.class);
```

Python

```
import logging

logging.basicConfig(level=logging.INFO)
logging.info("Hello")
```

40. Unit Testing

Java JUnit

```
@Test
public void testAdd() {
    assertEquals(5, add(2,3));
}
```

Python Pytest

```
def test_add():
    assert add(2,3) == 5
```

41. Decorators (Very Important)

```
def log(func):  
    def wrapper():  
        print("Before")  
        func()  
    return wrapper
```

Java Equivalent Concepts:

- AOP
- Proxy Pattern
- Interceptors

42. Context Managers

Python

```
with open("a.txt") as f:  
    data = f.read()
```

Java Equivalent:

```
try(resource) {  
}
```

43. Python Packaging Structure

```
project/  
|  
├─ app/  
|   ├── __init__.py  
|   ├── main.py  
|   └── service.py  
|  
└─ tests/
```

```
| requirements.txt
| README.md
```

44. Important Python Keywords

Keyword	Meaning
def	function
class	class
lambda	anonymous function
yield	generator
import	import package
from	partial import
pass	empty block
global	global variable
nonlocal	outer variable
async	asynchronous
await	wait async result
with	resource handling

45. Python Data Science Ecosystem

Package	Purpose
numpy	Numerical processing
pandas	Data analysis
matplotlib	Charts
seaborn	Visualization
scikit-learn	ML
tensorflow	Deep Learning
pytorch	Deep Learning

Package	Purpose
pyspark	Big Data

46. FastAPI Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello"}
```

47. Python for Data Engineering

Very important for your background.

Common Stack

- Python
- PySpark
- pandas
- Airflow
- FastAPI
- Databricks
- boto3

Example CSV Read

```
import pandas as pd

df = pd.read_csv("file.csv")
```

48. PySpark vs Java Spark

Java

```
Dataset<Row> df = spark.read().csv("a.csv");
```

Python

```
df = spark.read.csv("a.csv")
```

49. Python Memory Model

Important Differences:

Java	Python
Primitive vs Object	Everything is Object
Stack + Heap	Mostly Object References
JVM Memory	Python Runtime Memory
Strong Encapsulation	Flexible

50. Python Naming Conventions

Type	Convention
Variable	snake_case
Class	PascalCase
Constant	UPPER_CASE
Private	_name
Strong Private	__name

51. Common Interview Questions

Java Developers Learning Python

1. Difference between list and tuple
 2. What is GIL?
 3. Difference between deep copy and shallow copy
 4. What are decorators?
 5. Difference between multithreading and multiprocessing
 6. What is duck typing?
 7. What is monkey patching?
 8. Difference between Flask and Django
 9. What is asyncio?
 10. Difference between is and ==
-

52. Python Multiprocessing

```
from multiprocessing import Process
```

Used because of:

- GIL limitations
 - CPU intensive tasks
-

53. GIL (Global Interpreter Lock)

Very Important Concept.

Meaning:

- Only one thread executes Python bytecode at a time.
 - Multithreading helps mainly for I/O tasks.
 - Multiprocessing helps CPU-intensive tasks.
-

54. Pip Commands Cheat Sheet

```
pip install requests  
pip uninstall requests
```



```
pip list
pip freeze
pip freeze > requirements.txt
```

55. Useful Python Built-in Functions

Function	Purpose
len()	size
type()	datatype
range()	sequence
enumerate()	index + value
zip()	combine lists
map()	transformation
filter()	filtering
sorted()	sorting
sum()	total
max()	maximum
min()	minimum

56. enumerate and zip

```
names = ["a", "b"]

for index, value in enumerate(names):
    print(index, value)
```

```
a = [1,2]
b = [3,4]

for x, y in zip(a,b):
    print(x,y)
```

57. Python Slicing

```
name = "Python"

print(name[0:3])
print(name[-1])
print(name[::-1])
```

58. Copying Objects

```
import copy

copy.copy(obj)
copy.deepcopy(obj)
```

59. Dataclasses

Python

```
from dataclasses import dataclass

@dataclass
class Employee:
    id: int
    name: str
```

Java Equivalent:

- Lombok
- Records

60. Type Hints

```
from typing import List
```

```
def add(a: int, b: int) -> int:  
    return a + b
```

Equivalent to Java type declarations.

61. Python Project Setup

```
mkdir project  
cd project  
python -m venv venv  
pip install fastapi uvicorn
```

62. Recommended Learning Path for You

Phase 1 — Python Basics

- Variables
- Functions
- Collections
- OOP
- Exceptions
- Modules

Phase 2 — Advanced Python

- Decorators
- Generators
- Asyncio
- Multithreading
- Type Hints

Phase 3 — Data Engineering

- pandas
- PySpark
- Airflow
- Databricks
- boto3

Phase 4 — AI Integration

- LangChain
 - OpenAI APIs
 - RAG
 - Vector DBs
 - AI Agents
-

63. Java to Python Mental Mapping

Java Thinking	Python Thinking
Classes Everywhere	Functions + Simplicity
Strict Structure	Rapid Development
Enterprise Pattern Heavy	Minimalism
Boilerplate	Readability
Interfaces	Duck Typing
Streams	Comprehensions
DTOs	dict/dataclass

64. Common Mistakes by Java Developers

1. Writing too many classes
 2. Overusing getters/setters
 3. Ignoring list comprehensions
 4. Avoiding dynamic typing completely
 5. Writing Java-style verbose code
 6. Ignoring Python package ecosystem
 7. Misunderstanding indentation
 8. Forgetting mutable vs immutable types
-

65. High-Value Python Libraries for Your Career

AI + Data Pipeline Focus

Library	Use Case
pandas	CSV/Data Processing
pyspark	Big Data
requests	REST APIs
fastapi	APIs
sqlalchemy	Database ORM
boto3	AWS Integration
airflow	Scheduling
langchain	AI Agents
openai	LLM Integration
pydantic	Validation

66. Python vs Java Performance Reality

Area	Better Choice
Enterprise Backend	Java
AI/ML	Python
Rapid Prototyping	Python
High Throughput APIs	Java
Data Science	Python
Big Data	Both
Automation	Python

67. Final Quick Mapping Table

Java	Python
System.out.println	print
ArrayList	list
HashMap	dict
HashSet	set
Stream API	list comprehension
Exception	Exception
Thread	threading.Thread
CompletableFuture	asyncio
Spring Boot	FastAPI/Flask
Maven	pip
Lombok	dataclass
Jackson	json/pydantic
JUnit	pytest

68. Recommended Tools

Purpose	Tool
IDE	VS Code / PyCharm
Package Mgmt	pip / poetry
API Testing	Postman
Notebook	Jupyter
Formatting	black
Linting	pylint
Type Checking	mypy

69. Cheat Memory Tricks

Remember These

Python Uses:

- Indentation
- Simplicity
- Readability
- Dynamic Typing
- Everything as Object

Java Uses:

- Explicit Types
 - Boilerplate
 - Enterprise Structure
 - JVM Ecosystem
-

70. Your Best Career Combination

Your strongest future stack can be:

- Java + Spring Boot
- Python + AI/ML
- PySpark + Databricks
- FastAPI + AI Agents
- RAG + Vector Databases
- Multi-tenant Data Pipelines

This combination is extremely valuable in modern enterprise AI and data engineering ecosystems.

71. Recommended Practice Projects

Beginner

1. CSV Processor
2. REST API Client
3. File Automation Tool
4. JSON Transformer

Intermediate

1. FastAPI CRUD App
2. ETL Pipeline
3. PySpark Data Processor
4. S3 File Upload Service

Advanced

1. AI Agent for Metadata Extraction
2. RAG-based Pipeline Builder
3. AI-powered Data Mapper
4. Databricks Workflow Generator

72. Final Advice for Senior Java Developers

Do NOT try to replace Java with Python.

Instead:

- Use Java for enterprise-grade systems.
- Use Python for AI, automation, and data workloads.
- Combine both strategically.

Your 15 years of architecture and backend experience becomes significantly more valuable when combined with Python + AI ecosystems.

End of Cheat Sheet